

이중 스파이 딜레마 (Double Agent Dilemma)

- 시간: 12분
- 공간: 5 GB
- 환경: GPU 1개(≈16 GB VRAM), 인터넷 없음
- 솔루션크기: `solution.ipynb` ≤ 1 MB
- 베이스라인 점수: 0

두 컴퓨터 모델 Model R(ResNet-18)과 Model V(ViT-Tiny) 가 사진 분석 중, 두 모델은 100% 정확도를 기록하고 있으며, 모든 이미지에 대해 서로 같은 판단을 내리고 있는 중. 이 모델들이 얼마나 다른지 시험하기 위해 각 사진에 거의 눈에 띄지 않는 아주 작은 픽셀 변화를 가하여 Model R과 Model V가 완전히 다른 판단을 내리도록 하라



1. 과제

사전 학습된 두 이미지 분류기가 동일한 이미지 봄. 본 과제에서 제공되는 이미지들에 대해 두 분류기 모두 100% 정확도임.

- **Model R:** `torchvision.models.resnet18` (CNN, ResNet18).
- **Model V:** `timm`의 `vit_tiny_patch16_224` (Transformer, ViT-Tiny).

과제는 두 모델이 서로 다른 판단 내리도록 각 이미지에 대해 작은 변화를 만드는 것이며 모든 이미지에 대해 **서로 다른 두 개의** 변화를 만들어야함

- **Type A:** 이를 추가한 후 Model R은 여전히 이미지를 올바르게 분류하지만, Model V는 잘못 분류합니다.
- **Type B:** 이를 추가한 후 Model V는 여전히 이미지를 올바르게 분류하지만, Model R은 잘못 분류합니다.

각 변화는 알아채기 어려울 만큼 충분히 *작아야* 합니다. 작을수록 점수가 높음 (5절 참조). 변화는 원본 이미지에 픽셀 수준에서 직접 적용됨

2. 공개 데이터

이미지 집합이 제공됨. 두 개의 `split train`(이미지 100장) 및 `test_public`(이미지 100장) 으로 구성됨, 각 `split`에는 해상도 서로 다른 이미지들 포함되어 있음. 모든 이미지는 ImageNet-1K의 1000개 클래스에서 가져옴, Model R과 Model V는 두 `split` 에서 100% 정확

다음 파일들이 제공됨

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

채점 시 `dataset/test_public/` 폴더가 공식 채점을 위해 두 개의 비공개 이미지 집합 (`test_leaderboard_a` 및 `test_leaderboard_b`)으로 투명하게 대체됨. 각 집합은 PNG 형식의 **이미지 100장**과 레이블 파일을 포함함.

참고: 본 과제에서는 테스트 데이터셋의 레이블에 접근 가능함

3. 출력 형식

각 이미지에 대해 두 개의 파일을 생성해야함

```
{index}_a.pt  # Type A perturbation
{index}_b.pt  # Type B perturbation
```

`{index}` (0, 1, 2, ...)는 데이터셋에 있는 이미지 이름과 일치함. 각 파일은 `torch.save`으로 저장된 단일 텐서임. `shape`는 반드시 `3 x H x W`이어야 하며, 여기서 `H`와 `W`은 해당 이미지의 **원본** 해상도와 일치해야함 (224 x 224이 아님). 코드는 하나의 ZIP 파일 `submission.zip`만 생성해야함. 모든 `.pt` 파일을 ZIP 아카이브의 최상위 레벨에 두고, 감싸는 폴더, 하위 디렉터리가 없어야함

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

출력 형식에 문제가 있으면 노트북이 알려줌

4. 제약 조건

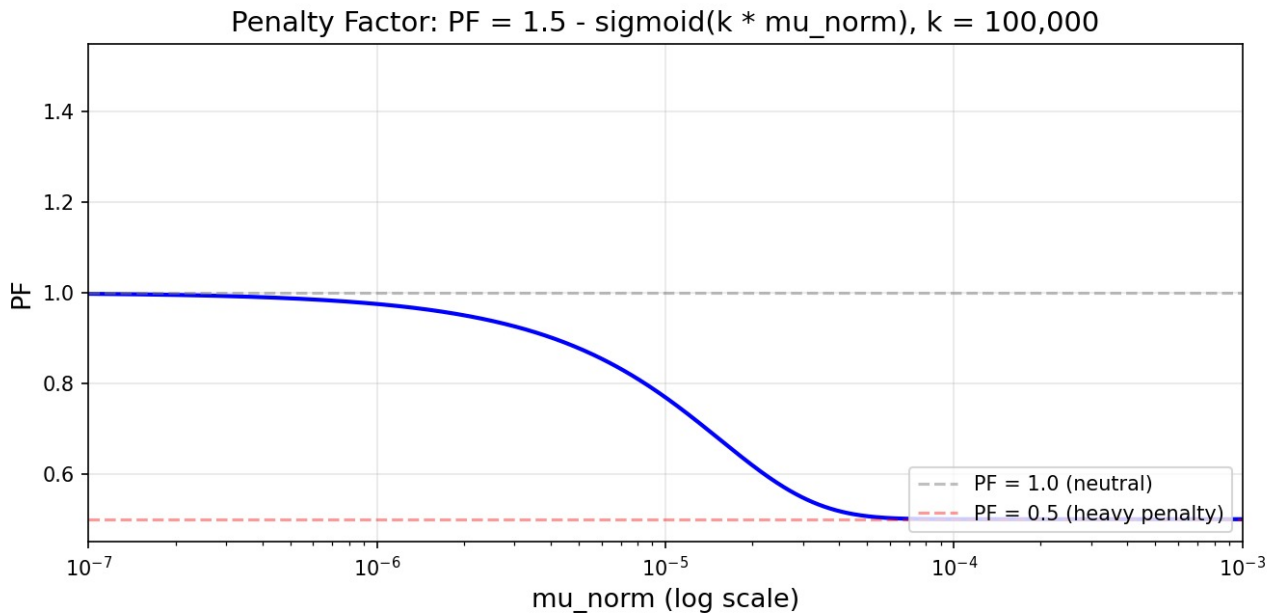
- **모델:** 반드시 `torchvision.models.resnet18(pretrained=True)` 과 `timm.create_model('vit_tiny_patch16_224', pretrained=True)` 을 사용해야함 다른 사전 학습 모델은 안됨
- **변환 파이프라인(평가 시 강제 적용):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])`. See Section 3 of `baseline.ipynb` 에서 자세한 내용을 확인할것
- **변화 해상도:** 원본 원시 이미지 해상도와 일치해야 합니다(224×224 가 아님). 텐서는 변환 파이프라인 *이전*에 원시 이미지에 더해짐
- **출력 형식:** .pt 파일만 허용됨 PNG/JPG는 안됨 텐서는 원시 이미지에 더해지며 전처리 전에 픽셀 값은 `[0, 1]`로 클리핑됨
- **파일 이름:** 평면 목록 형태, 엄격히 `{index}_a.pt / {index}_b.pt` 형식을 따름, zip 내부에 하위 디렉터리가 없어야함
- **라이브러리:** `torch, torchvision, timm`.

5. 채점

최종 점수 계산식임 M 을 해당 split의 이미지 수, $Score_A$ 을 성공한 Type A 변화의 수, $Score_B$ 를 성공한 Type B 변화의 수라고 하겠음

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF는 norm이 큰 변화에 벌점 주고 성능의 상한 근처에서 매우 민감하도록 설계된 함수임. 값은 0.5에서 1 사이의 범위로 제한됨 전체 구현은 `solution.ipynb`의 8절에서 확인.



벌점 함수 곡선

6. 제출물 확인

노트북에는 형식 문제 있을 시 알려주는 검사가 있으며, `solution.ipynb` 노트북의 7절에 있음

7. 로컬 테스트

`solution.ipynb`에는 완전하게 동작하는 예제가 있음. 공개 데이터, 두 모델, 공식 채점기를 불러오고 제출용 ZIP 파일을 작성함. 시작하기 전에 읽을 것.

8. 제출 방법

- 변경 사항을 `solution.ipynb`에 저장.
- JupyterLab 왼쪽 사이드바에서 Git 탭을 오픈.
- `solution.ipynb`을(를) **Stage** 할것 (+ 아이콘).
- 커밋 메시지를 입력하고 **Commit**을 클릭
- 위쪽 화살표가 있는 구름 아이콘을 클릭하여 푸시
- 이 Contest 페이지로 돌아와 **Submit**을 클릭

하나의 파일, `solution.ipynb`이라는 파일 제출할 것